AI Assisted Coding

I.HARSHITH

2303A51624

BT.NO: 08

Lab Assignment 16.3

Task 1: Database Schema Design Question

Design SQL schema for an online course platform with Instructors, Courses, and Students including relationships.

Prompt

"Generate SQL CREATE TABLE statements for instructors, courses, and students with primary keys, foreign keys, and normalized structure."

Code

```
CREATE TABLE Instructors (
instructor_id INT PRIMARY KEY,
name VARCHAR(100),     country
VARCHAR(50)
);
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
title VARCHAR(100),
instructor_id INT,
   FOREIGN KEY (instructor_id) REFERENCES Instructors(instructor_id)
);
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),     email
```

```
VARCHAR(100),     course_id
INT,
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Output

Tables created successfully.

Code Explanation

- Three tables created: Instructors, Courses, Students
- Primary keys uniquely identify records
- Foreign keys maintain relationships
- Schema follows basic normalization (no redundancy)

Task 2: Insert Sample Data

Question

Insert sample records into all tables and verify using SELECT.

Prompt

"Generate SQL INSERT statements for instructors, courses, and students with sample data."

Code

```
INSERT INTO Instructors VALUES
(1, 'John', 'USA'),
(2, 'Priya', 'India'),
(3, 'Lee', 'China');
INSERT INTO Courses VALUES
(101, 'Python', 1),
(102, 'DBMS', 2),
(103, 'AI', 3),
```

```
(104, 'ML', 2);

INSERT INTO Students VALUES
(1, 'Alice', 'alice@mail.com', 101),
(2, 'Bob', 'bob@mail.com', 102),
(3, 'Charlie', 'charlie@mail.com', 103);
SELECT * FROM Instructors;
SELECT * FROM Courses;
SELECT * FROM Students;
```

Output

```
Instructors:
1 John USA
2 Priya India
3 Lee China Courses:
101 Python 1
102 DBMS 2
103 AI 3
104 ML 2
Students:
1 Alice alice@mail.com 101
2 Bob bob@mail.com 102
3 Charlie charlie@mail.com 103
```

Code Explanation

- Inserts sample data into all tables
- SELECT verifies correctness
- Ensures relationships are valid

Task 3: CRUD Operations

Question

Perform retrieve, update, and delete operations.

Prompt

"Generate SQL queries for retrieving courses, updating student email, and deleting a course."

Code

```
-- Retrieve all courses
SELECT * FROM Courses;
-- Update student email
UPDATE Students
SET email = 'alice_new@mail.com'
WHERE student_id = 1;
-- Delete a course
DELETE FROM Courses
WHERE course_id = 104;
-- Verify
SELECT * FROM Students;
SELECT * FROM Courses;
```

Output

```
Courses:
101 Python
102 DBMS
103 AI
Students:
1 Alice alice_new@mail.com 101
```

Code Explanation

- SELECT retrieves data
- UPDATE modifies student email

- DELETE removes course
- Verification ensures correctness

Task 4: Aggregate Queries

Question

Generate reports using aggregate functions.

Prompt

"Generate SQL queries to count students per course and filter courses with more than 1 enrollment."

Code

```
-- Count students per course
SELECT course_id, COUNT(*) AS total_students
FROM Students
GROUP BY course_id;
-- Courses with more than 1 student
SELECT course_id, COUNT(*) AS total_students
FROM Students
GROUP BY course_id
HAVING COUNT(*) > 1;
```

Output course_id

total_students

101 1

102 1

103 1

(No course has more than 1 student)

Code Explanation

- GROUP BY groups students per course
- COUNT calculates total students
- HAVING filters aggregated results

Task 5: Query Optimization Question

Optimize query to fetch courses taught by instructors from a specific country.

Prompt

"Analyze and optimize SQL query using joins and indexing suggestions."

Code Original Query

```
SELECT title
FROM Courses
WHERE instructor_id IN (
  SELECT instructor_id FROM Instructors WHERE country = 'India'
);
```

Optimized Query

```
SELECT c.title
FROM Courses c
JOIN Instructors i ON c.instructor_id = i.instructor_id
WHERE i.country = 'India';
```

Index Suggestion

```
CREATE INDEX idx_instructor_country ON Instructors(country);
```

Output

DBMS

ML

Code Explanation

- JOIN replaces subquery → faster execution
- Index improves search performance
- Same result, better efficiency